

우드득(Wooduduk) — 프로젝트 기술 소개

장르: 장작패기 슬롯 서바이벌 × 실시간 멀티플레이(최대 4인)

엔진/기술: Unity 6 (URP) · C# · Firebase Realtime Database(서버리스)

작성: 한충희 (클라이언트 파트) — 본 문서는 우드득의 시스템 구조와 본인이 담당한 구현을 소개합니다.

핵심 성과 (한눈에)

- 전용 서버 없이 **Firebase**만으로 최대 4인 실시간 멀티 — 서버시간 동기 시작 · 호스트 마이그레이션
- 기존 코어를 한 줄도 수정하지 않고 카드 효과 10종 확장 — 흑 + 집계 전략 구조, 회귀 0건
- QA 전담 인원 없는 팀에서 자동 검증 체계 직접 구축(48종) — 밸런스 결함까지 출시 전 발견
- 티어/LP 정산 · 정적 이벤트버스 모듈화 · 결정론 시드로 재현 가능한 설계
- 재현이 어려운 트러블슈팅을 근본 원인까지 추적해 해결(멀티 4건 + 카드 밸런스 1건)

동봉 자료: `우드득_코드맵/index.html` — 실제 4개 씩(타이틀·튜토리얼·데브·멀티)에서 가동되는 라이브 코드 183개를 의존관계 그래프로 시각화한 인터랙티브 대시보드. (죽은 코드 66개는 제외)

0. 한눈에 보기

항목	내용
게임	슬롯머신을 돌려 장작을 모으고 모닥불로 체온을 유지하며 버티는 서바이벌. 최대 4인이 실시간으로 경쟁해 최후 생존/최고 점수로 순위를 겨루는 배틀.
장르·모드	캐주얼 스코어어택 · 실시간 멀티(최대 4인) · 싱글 생존
플랫폼·엔진	PC·모바일 / Unity 6 (URP) · C#
핵심 기술	Firebase Realtime Database 기반 서버리스 멀티, 서버시간 동기화 시작, 정적 이벤트버스, 결정론적 시드
본인 담당 파트	클라이언트 — 실시간 멀티플레이 세션 시스템, 티어·LP 정산, 슬롯 엔진 안정화·카드 시스템

프로젝트 이력 — 성장 과정

세 번의 프로젝트를 거치며 개인 → 팀으로, 기능 구현 → 시스템 설계로 발전했고, AI 활용도 웹 챗봇 → AI 코딩 에이전트로 확장됐습니다. 본 문서는 그 마지막인 우드득을 중심으로 다룹니다.

기간	프로젝트	형태	장르	내 역할	AI 활용
2026.03	Scene Thriller	개인	1인칭 사운드 퍼즐 공포 방 탈출	기획·구현 전반	웹 챗봇
2026.04~05	LuckyRandomDefense	팀	2D 랜덤 디펜스	팀장·PM·백엔드·레벨	웹 챗봇
2026.06~07	우드득(Wooduduk)	팀	실시간 멀티 스코어어택	클라이언트 시스템	AI 에이전트

- **Scene Thriller** *(개인)* — 리소스를 직접 수급해 방 텍스처와 퍼즐·상호작용 등 핵심 기능을 구현. 게임 하나를 끝까지 완성.
- **LuckyRandomDefense** *(팀)* — 팀장·PM으로 일정·WBS를 관리하고, **PlayFab** 기반 클라우드 세이브·밸런스 데이터 로딩(서버 데이터 백엔드)과 레벨 디자인을 담당. 이 PlayFab 경험이 우드득의 Firebase 실시간 데이터로 이어짐.
- **우드득** *(팀)* — 아래에서 상세히 다루는, 실시간 멀티 세션·티어 정산·슬롯 안정화를 맡은 팀 프로젝트.

1. 프로젝트 개요

1.1 게임 소개

플레이어는 눈 덮인 설원에서 슬롯머신(도끼 릴)을 돌려 **장작(재화)** 을 모으고, 그 장작을 **모닥불**에 넣어 계속 떨어지는 **체온**을 유지하며 생존한다. 체온이 0이 되면 사망. 슬롯 결과에 따라 배울·콤보·특수효과(막걸리)가 발동하고, 상점에서 **카드**를 사 슬롯 규칙을 강화한다.

싱글 생존에 더해, **최대 4인이 같은 방에서 실시간으로** 동시에 슬롯을 돌리며 서로의 점수·체온·생사를 지켜보고 최후까지 버티거나 최고 점수를 내 **순위(랭킹)** 를 겨루는 것이 핵심 재미다. 순위 결과는 **티어/LP**로 누적되어 시즌 랭크가 오르내린다.

인게임 화면



게임 HUD — 체온·슬롯·콤보·카드



설원의 모닥불 — 체온을 유지하는 생존 무대



실시간 멀티 세션 — 이탈·사망·최후생존 처리(본인 파트)



슬롯머신 — 도끼 릴로 장작을 모으는 핵심 루프

1.2 씬 흐름

Title(로그인/타이틀) → Tutorial(튜토리얼) → Dev(싱글 플레이) └─▶ 결과/정산 → (재도전/로비)
Multi(실시간 4인 대전) └─▶

- **Title** — Firebase 인증 로그인, 유저 데이터 로드, BGM/설정.
- **Tutorial** — 슬롯·모닥불·체온 규칙 학습(연출 스크립트 구동).
- **Dev(싱글)** — 혼자 생존 플레이. 매칭 없이 즉시 시작.
- **Multi** — 매칭으로 방을 잡아 4인 실시간 대전. 카운트다운 후 전원 동시 시작.

1.3 팀 구성과 담당 경계

여러 명이 분업한 팀 프로젝트이며, 시스템 소유권을 명확히 나눠 병합 충돌을 최소화했다. *(팀 인원/기간 등 세부는 제출 시 확인)*

- **한충희(본인)** — 클라이언트: 멀티 세션 시스템(동기 시작·좌석·라이브 동기·라이프사이클·결과 정산), 티어/LP 정산, 슬롯 엔진 안정화·카드.
- **네트워크/백엔드 담당** — Firebase 매칭 코어(RoomRepository/RoomSession/MatchmakingFlow), 방 스키마.
- **슬롯 엔진 담당** — 슬롯 추천/판정/생존 처리 코어(SlotMachine/SurvivalProcessor).
- **UI·아트 담당** — 타이틀/결과/티어 UI, 모델링·애니메이션 씬.

협업 원칙: 타 담당자 소유 파일은 **추가(additive)** 변경만 하고 사전 공유. 공용 상태는 정적 이벤트로 디커플링해 서로의 코드를 직접 건드리지 않도록 설계.

2. 구현 원리 (핵심 시스템)

2.1 실시간 멀티플레이 — Firebase 서버리스

전용 게임서버(권위 서버) 없이 **Firebase Realtime Database**를 **공유 상태 저장소**로 사용해 실시간 대전을 구현했다. 각 클라이언트가 자기 상태를 DB에 쓰고 상대 상태를 구독해 읽는 **P2P적 동기화** 구조다.

(1) 매칭 & 고스트 백필

- `activeRooms · rooms/{roomId}` 노드로 대기방을 탐색/생성한다. 방 생성은 방+첫 플레이어를 **원자적(UpdateChildren)**으로 기록해 "빈 방 노출" 레이스를 제거했다.

- 접속 종료 시 `onDisconnect().RemoveValue()` 로 방/플레이어/활성표시를 자동 정리해 유령 방을 방지한다.
- 정원이 안 차면 호스트가 타임아웃 후 **고스트(AI 대역)** 로 남은 자리를 채운다. 고스트는 로컬에서 시뮬레이션되므로, 상대가 부족해도 항상 4인 대전 경험을 보장한다.

(2) 서버시간 동기화 시작 — 전원 "같은 순간" 발화

멀티에서 가장 중요한 문제는 "*"로딩 속도가 다른 여러 클라이언트를 어떻게 동시에 출발시키나"였다. 로컬 시계는 서로 다르므로 이를 해결하기 위해:

1. `.info.serverTimeOffset` 로 각 클라이언트가 자기 시계와 서버 시각의 오차를 구한다.
2. 호스트가 `rooms/{roomId}/startAt` 에 **절대 서버 시각(ms)** 을 기록한다.
3. 전원이 그 절대 시각까지 각자 카운트다운 → **같은 순간** 게임을 발화한다.

발화 시점에 체온 드레인이 시작되고 스피ن 잠금이 풀린다. 호스트 선출은 **접속 중인 실플레이어 중 최소 uid**를 결정론적으로 계산해, 별도 합의 통신 없이 모든 클라이언트가 같은 호스트를 뽑는다.

(3) 좌석 표시 & 라이브 동기

- 상대 좌석은 **표시 전용 헬로** 구성했다. 실제 슬롯머신 컴포넌트를 붙이면 내 스피ن 이벤트에 상대 슬롯이 전부 반응하는 문제가 있어, 좌석 뷰는 **전역 슬롯 이벤트를 구독/발신하지 않고** 상대의 라이브 데이터(점수·콤보·체온·생사·모닥불)만 폴링해 자기 비주얼을 갱신한다.
- 각 클라이언트가 자기 상태를 주기적으로 DB `live/{uid}` 에 쓰고, 상대들 것을 구독해 좌석/상태판에 반영한다.

(4) 세션 라이프사이클 — 이탈·사망·최후생존·호스트 마이그레이션

- 로스터 변화를 구독해 누가 이탈했는지 감지하고, 호스트가 이탈자의 사망을 **대필(역등)** 처리해 순위 계산이 누락되지 않게 했다.
- 호스트가 나가면 남은 실플레이어 중 최소 uid가 자동으로 새 호스트가 된다(마이그레이션).
- 사망 수가 정원-1에 도달하면 마지막 생존자를 1위로 확정(최후생존 종료).

(5) 결과 정산 & 실시간 랭킹

- 진행 중 각자의 점수로 실시간 순위 배지를 갱신하고, 종료 시 확정 순위를 받아 **티어/LP·재화**를 정산한다. 결과 UI는 모닥불 앞에 표시되고 페이드로 매칭 화면에 복귀한다.

2.2 슬롯 엔진 & 생존 루프

- **결정론적 시드**: 방 id를 해시해 모든 클라이언트가 같은 시드를 갖는다. 첫 스피ن만 공유 시드로 동일 결과를 내고, 이후 각자 분기해 개인 플레이가 된다.
- **정적 이벤트버스(SlotEvent/CardEvent)**: 스피ن 시작·릴 결정·판정·효과·정산·사망을 이벤트로 브로드캐스트해 슬롯 로직과 연출/사운드/UI를 완전히 디커플링했다.
- **체온 생존**: 고정 틱으로 체온을 드레인하고, 모닥불 연소 중에는 회복. 정산 횟수에 따라 드레인이 가속되어 후반 난이도가 오른다.

2.3 티어 / LP 정산

- 순수 정적 로직(결정론·무부작용)으로 티어 승강을 계산한다. 같은 입력 → 같은 출력.

- 멀티는 **등수별 차등 보상**: 1등은 최대 획득, 2등은 절반, 중위권은 유지, 하위권은 감소. 싱글은 목표 점수 대비 성패로 판정.
- 승급 임계·강등 보호·시즌 소프트리셋 혹까지 규칙으로 정리해 밸런싱 수치를 코드 변경 없이 조정 가능하게 했다.

2.4 카드 시스템 — 코어를 수정하지 않는 혹 기반 확장

상점에서 카드를 사면 슬롯 규칙(추가 릴, 배울 보정, 정산 개입, 심볼 변환 등)이 런타임에 적용된다. 이 시스템은 **기존 판정 엔진을 건드리지 않고 콘텐츠를 계속 엮을 수 있는 구조**로 설계했고, 실제로 신규 카드 10종(C009~C018)을 그 위에 추가했다.

(1) 혹(hook) + 집계 전략(aggregation) 구조

각 카드는 `SlotEffect` 를 상속해 자신이 개입할 지점만 오버라이드한다. 예: 체온 드레인 배울(`TempDrainMultBp`), 정산 금액 변형(`ModifySettleAmount`), 심볼 재라벨(`RelabelSymbols`) 등. 슬롯 코어는 활성 카드 전체를 순회하며 이 혹들을 **집계**해 최종값을 낸다.

핵심은 **집계 방식을 카드가 아니라 코어가 정한다**는 점이다. 같은 종류의 효과가 여러 장 겹쳤을 때 어떻게 합쳐질지를 전략으로 분리했다.

집계	해당 혹 예시	겹쳤을 때
곱(product)	장작 배울, 체온 드레인	서로 곱해짐
합(sum)	콤보 추가	더해짐
최솟값(min)	실패 시 손실률	가장 유리한 값 (중첩 악용 방지)
순차(sequential)	정산 코인플립, 심볼 변환	차례로 적용

이 구조 덕에 **기존 판정 엔진과 기존 카드 8종을 한 줄도 수정하지 않고, 새 파일도 만들지 않고 10종을 추가**할 수 있었다. 카드를 추가하는 동안 기존 기능의 회귀는 0건이었다.

(2) 순서 의존성 제거 — 충돌하는 효과의 결정론 보장

심볼 자체를 바꾸는 카드 두 종(좀벌레↔스윙 변환, 팡→스윙 확률 변환)은 적용 순서에 따라 결과가 달라질 위험이 있었다. 이를 각 카드가 **변형된 중간 결과가 아니라 원본 스냅샷만 읽도록** 설계해, 어떤 순서로 적용해도 같은 결과가 나오도록 했다. 순서 의존성을 없애는 것이 이런 종류의 버그를 원천 차단하는 방법이었다.

(3) 자동 검증 체계 구축

QA 전담 인원이 없어 팀원이 각자 QA를 하는 환경이라, 수동 확인만으로는 회귀를 놓친다고 판단했다. 테스트 프레임워크가 도입되지 않은 프로젝트여서, **에디터 메뉴에서 실행하는 자체 검증 체계**를 만들어 카드 효과 48종 케이스를 자동 검증하도록 했다. 이 검증이 아래 §4 사례 5의 밸런스 결함을 출시 전에 드러냈다.

3. 기술 채택 사유

Firestore Realtime Database (서버리스) — vs 전용 게임서버

초기에 FishNet 기반 권위 서버를 검토했으나, 신입 규모 팀에서 서버 운영·배포·상태 동기 비용이 과했다. 대신 Firestore의 실시간 DB + `onDisconnect` 프레즌스 + 서버시각 오프셋만으로 **운영 부담 없이** 실시간 대전을 구

현했다. (구 FishNet 네트워킹 코드는 전환 후 폐기 — 코드맵에서 죽은 코드로 확인 가능)

정적 이벤트버스

슬롯 1회 스핀이 연출·사운드·UI·생존·네트워크 등 다수 시스템을 건드린다. 직접 참조로 엮으면 담당자 간 파일이 뒤섞여 병합 충돌이 잦아진다. 이벤트로 디커플링해 **각 담당이 자기 구독자만 관리**하도록 했다.

서버시간 동기화 시작

클라이언트 로컬 시계는 신뢰할 수 없다. 절대 서버 시각(startAt)을 기준으로 삼아 로딩 편차와 시계 오차를 흡수하고 **전원 동시 발화**를 보장했다.

결정론적 시드 + 고스트 백필

같은 시드 → 같은 슬롯 결과라 검증/재현이 쉽고, 상대가 부족한 시간대에도 고스트로 대전 경험을 유지해 매칭 품질 문제를 완화했다.

4. 트러블슈팅 & 회고

프로젝트 마무리 단계에서 실제로 추적해 해결한 대표 사례들이다. *(문제 → 원인 → 해결)*

사례 1 — 고스트가 "호스트"로 선출되어 게임이 시작 안 됨

- **문제:** 특정 유저만 멀티가 시작되지 않고 무한 대기. 재현이 불규칙했다.
- **원인:** 호스트를 "가장 작은 uid"로 뽑는데, 후보에 **고스트(uid ghost_...)**가 포함되어 있었다. 실플레이어 uid가 사전순으로 ghost_ 보다 뒤면 고스트가 호스트로 잡히고, 고스트는 시작 시각(startAt)을 기록하는 로직을 돌리지 않아 실플레이어가 영원히 대기했다.
- **해결:** 호스트 선출에서 **고스트를 제외**(실플레이어만 대상). 추가 안전망으로 "실플레이어가 하나도 없는 고아 방"은 방을 파기하도록 가드. uid 사전순에 따라 되고 안 되던 현상이 완전히 사라졌다.

사례 2 — 일부 플레이어가 체온이 이미 깎인 채로 시작

- **문제:** 어떤 플레이어는 게임 시작 순간 체온이 크게 떨어진 상태로 로드됐다.
- **원인:** 슬롯 Spin() 이 무조건 "**게임 시작(Started)**" 플래그를 켜는 싱글 플레이 전제를 갖고 있었다. 멀티에서는 카운트다운/시작 대기 중 스핀이 한 번 들어가면 그 순간부터 체온이 드레인되어, (호스트 로딩이 느려 대기가 길수록) 정식 시작 시 이미 체온이 낮았다. 드레인 속도로 역산하면 "한 번의 카운트다운"이 아니라 **수십 초 조기 드레인**이 원인임을 확인했다.
- **해결:** 런에 **AutoStartOnSpin** 플래그를 도입해 **멀티에서는 스핀이 시작 플래그를 켜지 못하게** 하고, 시작 소유권을 서버시간 동기 코디네이터로 일원화했다. 이제 정식 시작 전에는 체온이 절대 깎이지 않는다. (근본 원인 제거 방식으로, 우회 잠금보다 재발 가능성을 없앴)

사례 3 — 1:1에서 이겼는데 LP가 오히려 깎임

- **문제:** 멀티 1:1에서 이겨도 LP가 -로 정산됐다.
- **원인:** 티어 정산이 **순위를 무시하고 점수만** 보고 있었다. 승패 정보(등수)가 정산에 전달되지 않았다.
- **해결:** 정산에 **등수·인원 수**를 전달하고, 등수별 차등(1등 최대, 하위 감소) 규칙으로 교체했다.

사례 4 — 릴 추가 카드 중복 시 무한 루프 위험

- **문제:** 릴을 추가하는 카드가 상한 없이 중첩되면 릴 수가 폭주해 업데이트 루프가 걸릴 수 있었다.

- **해결: 3중 방어** — ① 카드 중복 구매 가드 ② 스펀 시 릴 수 상한 클램프 ③ 애니메이션 단계 클램프. 어느 한 곳이 뚫려도 다음 방어선이 잡도록 했다.

사례 5 — 카드를 겹쳐 사면 효과가 폭발하는 구조를 출시 전에 발견

- **문제:** 신규 카드 10종을 추가한 뒤 자동 검증(\$2.4)을 돌리던 중, 같은 카드를 중복 구매하면 곱으로 집계되는 효과가 지수적으로 증폭되는 것을 발견했다. 예: 체온 드레인 감소 카드 5장이면 사실상 무적, 배울 카드 5장이면 243배.
- **왜 드러나지 않았나:** 카드 가격이 구매마다 크게 올라 **가격이 중첩을 우연히 막고 있었다**. 밸런스 값을 한번만 조정해도 그대로 터질 잠재 결함이었다. 테스트로 가격을 낮추자 즉시 재현됐다.
- **분석·제안:** 카드 18종 전체의 1~5장 중첩 결과를 계산해 문서로 정리하고, **구매 시점 한 곳(TryBuyCard)만 수정하면 되는 상한(MaxStack) 방식**을 제안했다. 릴 카드에 이미 있던 상한 가드와 같은 자리라 구조 변경이 없다. 상한 수치는 기획 결정 사항으로 남겨 팀에 넘겼다.
- **배움:** 검증 체계는 버그만 잡는 도구가 아니라 **설계·밸런스 결함까지 드러낸다**. 그리고 "지금 문제가 없다"가 "구조가 안전하다"는 뜻은 아니라는 것을 확인했다.

회고

전용 서버 없이 실시간 대전을 만들며 **"권위가 없는 분산 환경에서 어떻게 일관성을 만드나"** 를 몸으로 배웠다. 서버시간 기준 동기화, 결정론적 선출/시드, 맥등 처리(이탈 대필), onDisconnect 프레즌스 같은 기법이 그 답이었다. 특히 버그를 "증상 우회"가 아니라 **근본 원인까지 추적해 제거**하는 습관(사례 2의 소유권 일원화)이 재발을 막는다는 걸 확인했다. 정적 이벤트버스와 소유권 분리 설계 덕에 다인 협업에서도 병합 충돌을 낮게 유지할 수 있었다.

본 문서의 시스템 구조는 동봉한 코드맵 대시보드([우드득_코드맵/index.html](#))에서 파일 단위 의존관계로 직접 탐색할 수 있습니다.